Cristaux Jewellery

Observability Project Report

Part A: Your Project

1. Problem Statement & Solution

Problem:
When running an online store, it is hard to know if the website is running slow, if orders are failing, or if the system is crashing until customers start complaining. Without a tracking system in place, finding and fixing these technical issues takes a long time.

Solution:
Cristaux Jewellery is an online store application built using Python (FastAPI). It lets customers browse luxury jewellery items, add them to a shopping cart, place orders, and cancel orders. To monitor everything happening in the background, the application is connected to tracking tools (Prometheus and Grafana for metrics, and Filebeat, Elasticsearch, and Kibana for logs). All these services run together smoothly inside Docker containers.

2. How to Run & Try the Application

Launch the containers:

```
docker compose up -d --build
```

Access the interfaces:

Service	URL	
Storefront Web UI	http://localhost:8000	
Application Metrics	http://localhost:8000/metrics	
Prometheus UI	http://localhost:9090	
Grafana Dashboards	http://localhost:3000	
Kibana Log Analytics	http://localhost:5601	

Simulate user traffic:

```
py -3 seed_traffic.py
```

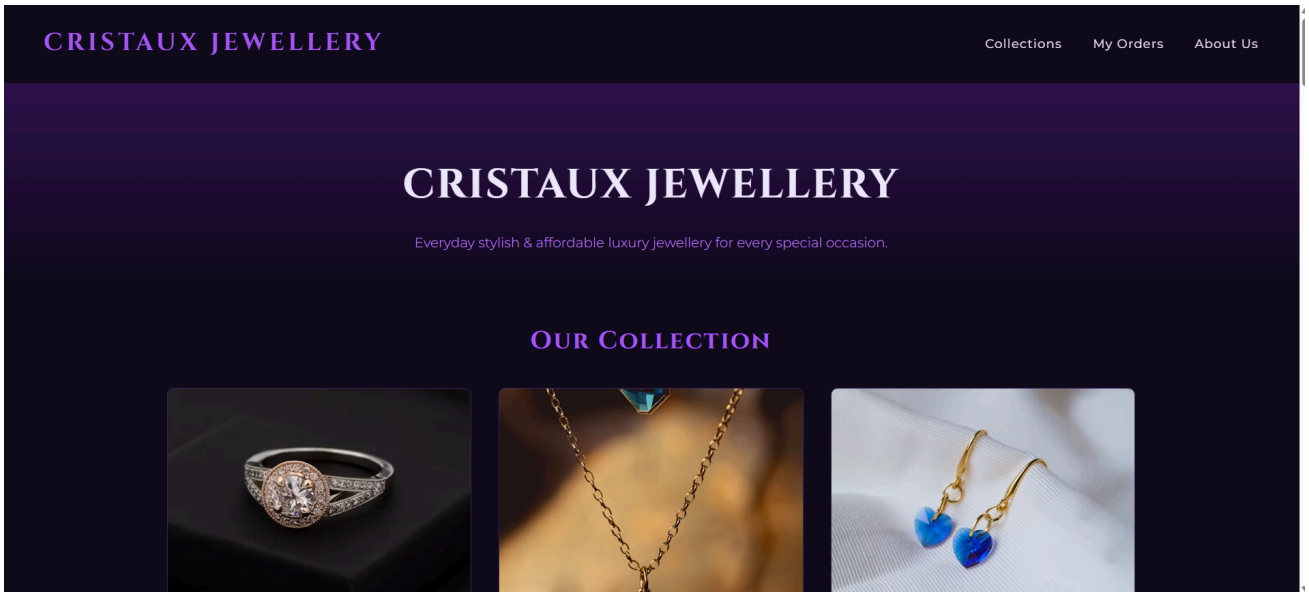


Figure 1

Part B: Metrics

1. Prometheus Metrics Table

The application exposes all four Prometheus metric types across business and application categories.

Metric Name	Type	Unit	Labels	Description / Code Location
`jewellery_orders_placed_total`	Counter	integer	`category`	Cumulative number of orders placed. Incremented in `place_order()` in `main.py`.
`jewellery_orders_cancelled_total`	Counter	integer	`reason`	Cumulative number of cancelled orders. Incremented in `cancel_order()` in `main.py`.
`jewellery_pending_orders_count`	Gauge	integer	None	Orders waiting to be fulfilled. Increments on order placement; decrements on fulfillment or cancellation.
`jewellery_active_carts_count`	Gauge	integer	None	Number of active shopping carts in memory.
`http_request_duration_seconds`	Histogram	seconds	`method`, `path`, `status`	Bucketed HTTP request durations (0.01 s to 5.0 s). Recorded in `observability_middleware()`.
`jewellery_order_processing_seconds`	Summary	seconds	None	Summarizes execution time for order checkout processing.

2. Grafana Queries & Dashboard Explanation

Panel	Metric Type	PromQL Query	Explanation
Orders Placed Rate	Counter	`rate(jewellery_orders_placed_total[1m])`	Rate of new orders per second, broken down by jewellery category.
Pending Orders Count	Gauge	`jewellery_pending_orders_count`	Live count of unfulfilled orders. Goes up when orders arrive and down when they are processed or cancelled.
95th Percentile Latency	Histogram	`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[1m])) by (le))`	95th percentile request duration, in seconds.
Node Exporter CPU Usage	Node Exporter	`100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)`	System CPU usage of the host machine.

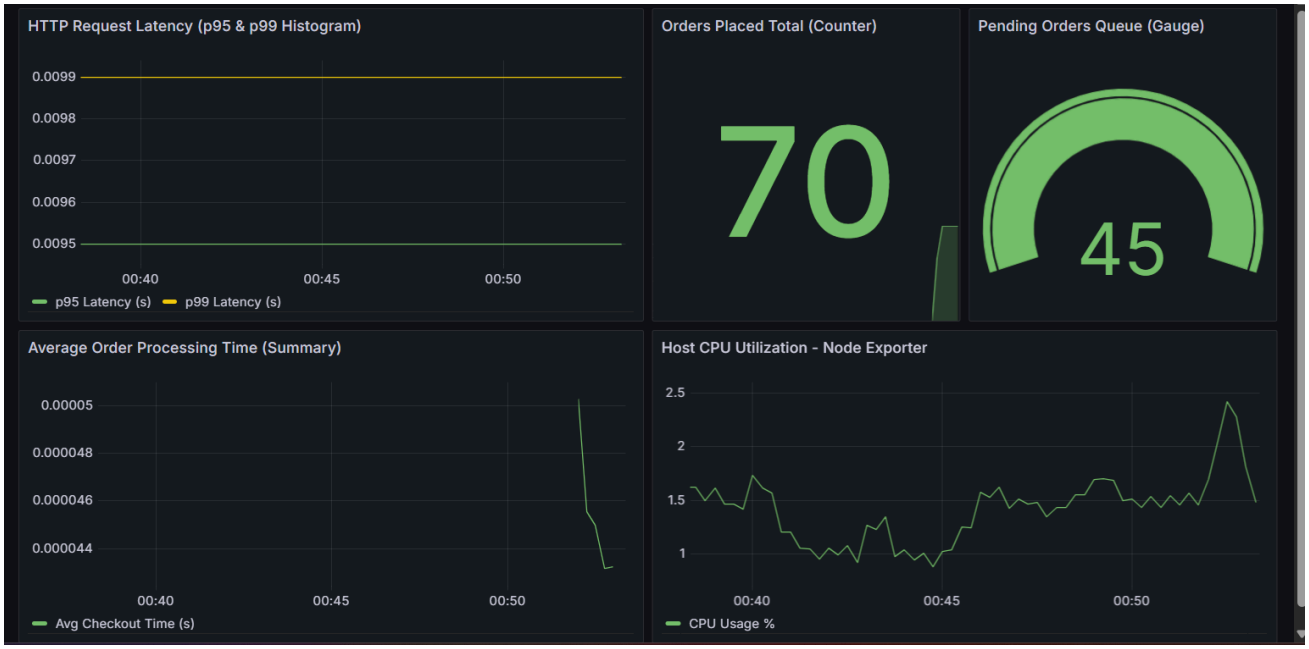


Figure 2

Part C: Logs

1. Logging Pipeline Architecture

The structured logging pipeline follows this path:

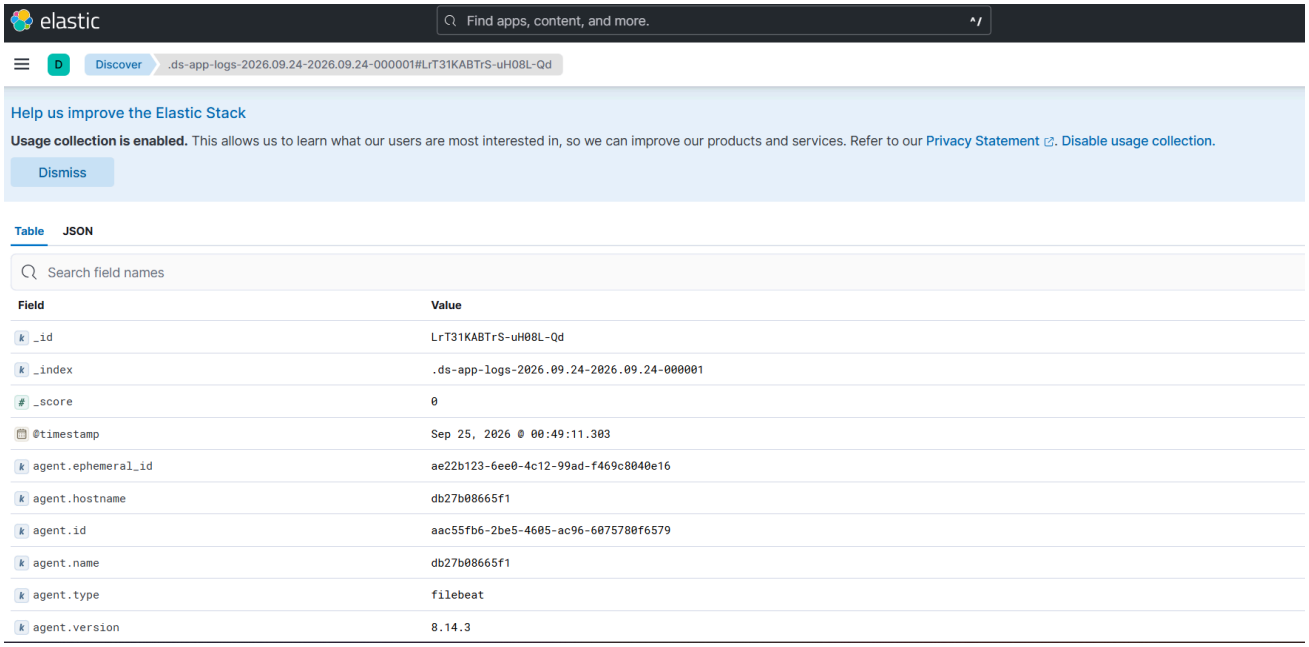
```
FastAPI App (JSON stdout) → Docker Logs → Filebeat → Elasticsearch → Kibana
```

- **Application level:** `main.py` uses a custom `JsonFormatter` that emits JSON records containing `@timestamp`, `service`, `level`, `message`, `request_id`, `path`, `status_code`, and `duration_ms`.
- **Collection:** Filebeat monitors container stdout log files on the host disk, parses the JSON payload, and ships events to Elasticsearch (`http://elasticsearch:9200`).

- **Storage & search:** Elasticsearch indexes the fields into `app-logs-*` indices. Kibana provides the UI for searching and filtering.

2. Sample JSON Log Record


```
{
  "@timestamp": "2026-09-24T12:00:00.123Z",
  "service": "Cristaux Jewellery",
  "level": "INFO",
  "message": "request processed",
  "request_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "method": "POST",
  "path": "/orders",
  "status_code": 200,
  "duration_ms": 14.52
}
```

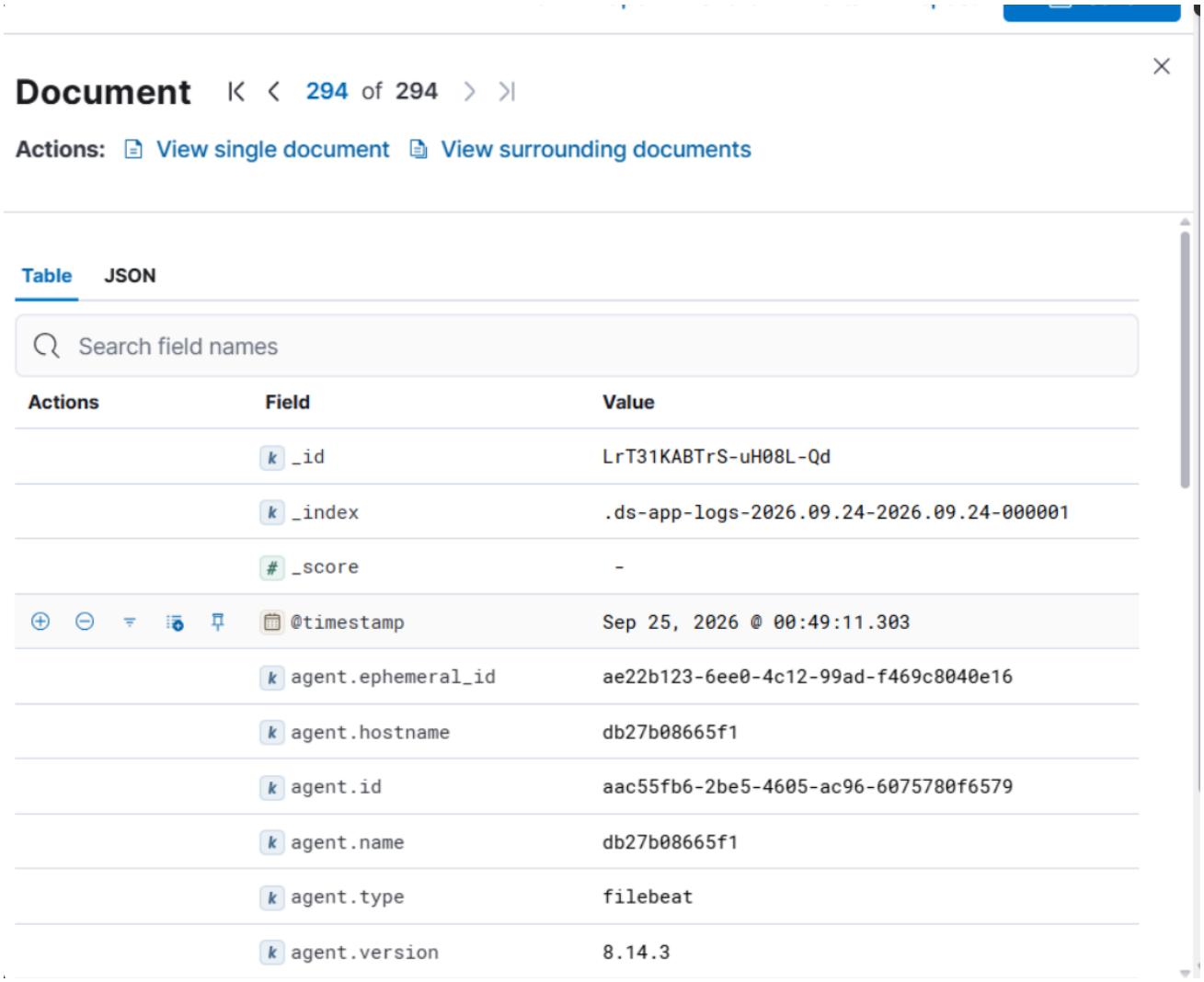
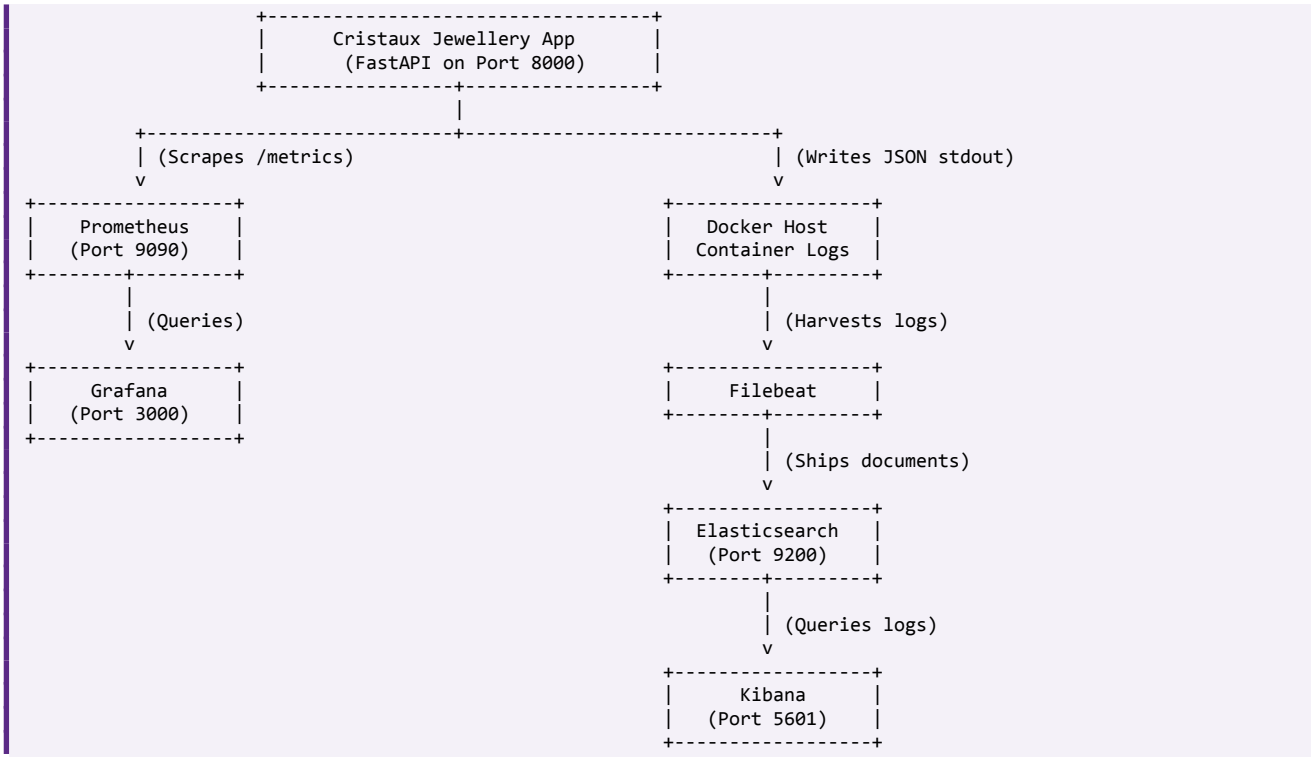

Document
294 of 294
Actions: View single document View surrounding documents
Table JSON
Search field names
Actions
Field
Value
_id
LrT31KABTrS-uH08L-Qd
_index
.ds-app-logs-2026.09.24-2026.09.24-000001
_score
-
@timestamp
Sep 25, 2026 @ 00:49:11.303
agent.ephemeral_id
ae22b123-6ee0-4c12-99ad-f469c8040e16
agent.hostname
db27b08665f1
agent.id
aac55fb6-2be5-4605-ac96-6075780f6579
agent.name
db27b08665f1
agent.type
filebeat
agent.version
8.14.3

Figure 3

Part D: System Design

1. Architecture Diagram



Component failures:

- If Filebeat or Elasticsearch stops working, log messages remain buffered in stdout and the Docker container log files, and the main application is not affected.
- If Prometheus goes down, metric scraping pauses, but the application continues to serve traffic normally.

2. Metric & Log Step-by-Step Path

Metric path

1. A user calls `POST /orders`.
2. `main.py` runs `ORDERS_PLACED_TOTAL.labels(category="rings").inc()`.
3. Prometheus scrapes `http://localhost:8000/metrics` every 15 seconds and updates its time-series database.
4. Grafana runs `rate(jewellery_orders_placed_total[1m])` to plot order velocity over time.

Log path

1. Middleware assigns a unique UUID to `x-request-id`.
2. `logger.info("request processed", extra={...})` writes JSON to standard output.
3. Filebeat reads the JSON string, parses the top-level fields, and posts it to Elasticsearch (`app-logs-*`).
4. The user searches `request_id: "9b1deb4d..."` in Kibana to trace the request's lifetime.

Part E: Experiments

Experiment E1: Reproduce a Latency Anomaly

- **Baseline:** Under normal traffic (`seed_traffic.py`), p95 response time stays below 50 ms.
- **Fault injection:** An artificial delay was introduced:

```
curl -X POST http://localhost:8000/admin/fault -H "Content-Type: application/json" -d "{\"delay_ms\": 500}"
```

- **Observed behavior:**
  - **Metrics:** The Grafana p95 latency chart spiked from 0.02 s to 0.52 s.
  - **Logs:** Kibana showed `duration_ms` rising to about 500 ms for incoming endpoints.
- **Recovery:** After setting `delay_ms` to `0`, p95 latency returned to baseline within two scrape intervals.

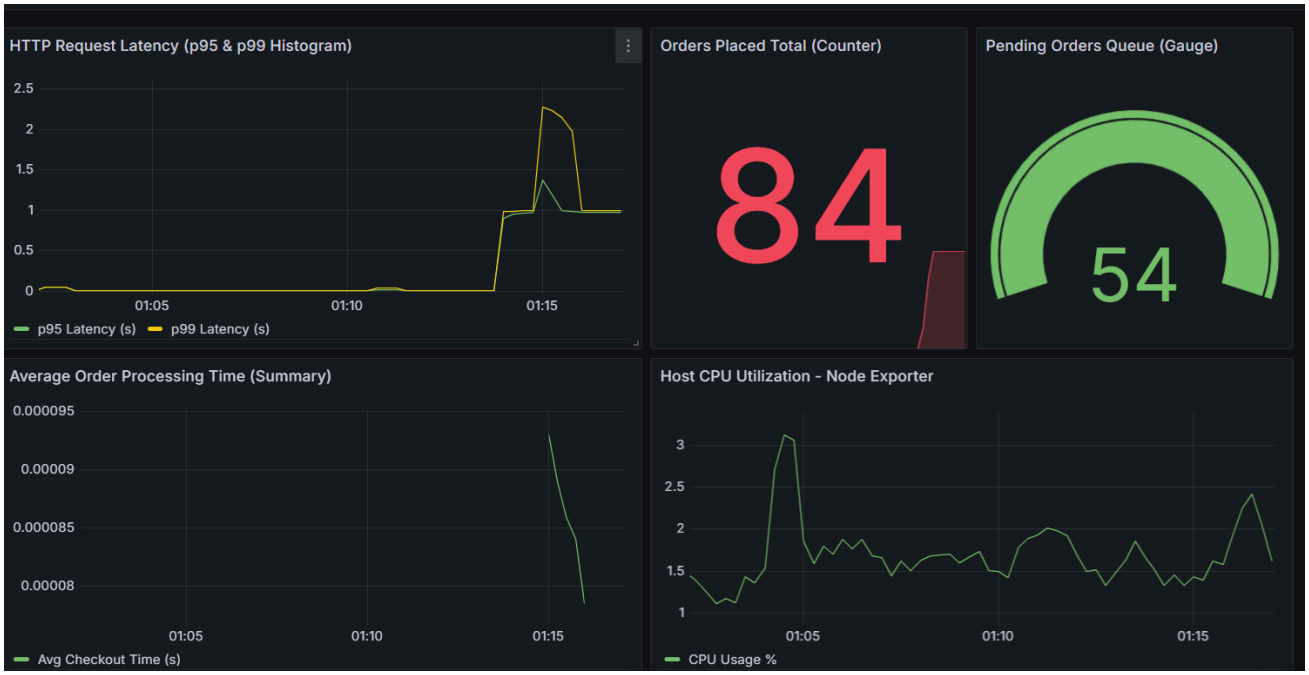


Figure 4

Experiment E2: Cardinality Explosion

- **Procedure:** Called `GET /admin/cardinality-test` repeatedly, attaching a unique `request_id` as a metric label on `demo_requests_total`.
- **Observations:** Running `count(demo_requests_total)` in Prometheus showed that each unique `request_id` value created a new time series in memory.
- **Analysis & conclusion:** High-cardinality values (UUIDs, timestamps, email addresses) cause cardinality explosions in Prometheus, consuming excessive memory and CPU during query evaluation. Unique identifiers belong in structured logs (Part C). Prometheus labels should be limited to finite, low-cardinality values such as HTTP status codes, request methods, and categories.

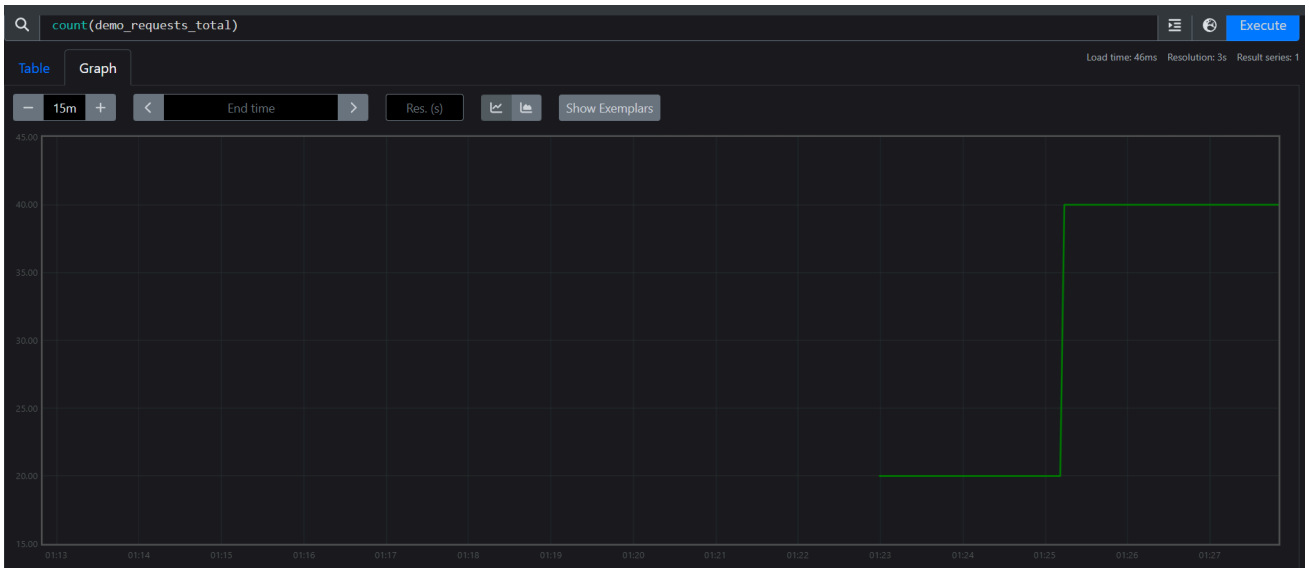

count(demo_requests_total)
Execute
Table
Graph
Load time: 46ms
Resolution: 3s
Result series: 1
15m
End time
Res. (s)
Show Exemplars
45.00
40.00
35.00
30.00
25.00
20.00
15.00
01:13
01:14
01:15
01:16
01:17
01:18
01:19
01:20
01:21
01:22
01:23
01:24
01:25
01:26
01:27

Figure 5